SAE : Sécurisation d’un broker MQTT

Par : ALLOUT Sirine & LADLANI Lydia

JANVIER 2026

Groupe : TP C1

Encadré par : Mr LAVAL

1. Contexte du projet et démarche de sécurisation des communications

Dans ce projet, le but est de mesurer une température réelle avec une sonde **Pt100** puis de la rendre exploitable par un **ESP 32** : comme la Pt100 ne fournit qu'une variation de résistance relativement faible, on met en place une **chaîne de conditionnement** (courant constant ≈ 1 mA pour convertir la résistance en tension, câblage **3 fils** pour compenser l'influence des résistances de ligne, puis **adaptation et amplification** via AOP avec filtrage éventuel) afin d'obtenir un signal stable, peu bruité et correctement dimensionné pour l'entrée **ADC 0 - 3,3 V** du microcontrôleur.

Donc, une fois la donnée acquise et convertie, on la transmet vers Internet au sein d'une **cellule Wi-Fi** en **mode infrastructure**, identifiée par un **SSID** (nom du réseau) et un **BSSID** (identifiant matériel du point d'accès), puis vers un **broker MQTT** selon un modèle **client/serveur** : l'ESP 32 est un **client publisher** qui publie des messages sur un **topic**, tandis qu'un PC ou un téléphone devient **client subscriber** en s'abonnant à ce topic pour recevoir les valeurs.

C'est là que la **cybersécurité** intervient, car il faut garantir la **confidentialité** (éviter l'écoute des mesures), l'**intégrité** (empêcher une modification discrète des messages) et l'**authentification/contrôle d'accès** (s'assurer que seuls les bons équipements peuvent publier ou lire), ce qui correspond à la **triade CIA** appliquée à un système IoT. Les risques concrets associés sont typiques : attaques sur identifiants (**brute force** en testant toutes les combinaisons, ou **dictionnaire** avec des mots de passe courants), attaques sur la disponibilité (**DoS/DDoS** en saturant un service ou un réseau pour empêcher la communication), et attaques d'interception comme le **MITM** où un attaquant se place entre client et serveur pour écouter, modifier ou rediriger le trafic.

Pour y répondre, on fait évoluer progressivement la communication à travers des **niveaux de sécurité**, en changeant notamment de **ports** et de mécanismes : de **MQTT simple** (1883, échanges en clair) vers une **authentification par login/mot de passe** (1884, contrôle d'accès mais toujours sans chiffrement), puis **MQTT sur TLS** (8883) où un tunnel chiffré protège confidentialité et intégrité et où le client vérifie l'identité du serveur grâce à une **CA** et à son certificat, et enfin le **mTLS** (8884/8885) où serveur et client s'authentifient mutuellement via **certificat client + clé privée** (preuve d'identité côté ESP32). Tout cela est négocié lors du **handshake TLS**, étape de "poignée de main" qui établit les clés de session et valide les certificats ; et comme la validité des certificats dépend du temps, l'ESP32 doit synchroniser son horloge via **NTP** afin d'éviter des échecs de connexion liés à une date incorrecte.

2. Résultats par étape

2.1 Port 1883 - MQTT non sécurisé

But : publier des messages et les voir côté client (MyMQTT).
Configuration : host test.mosquitto.org, port **1883**, topic **geiiiutv/syrine/n1.**
Sécurité : aucune, tout passe en clair.

Réalisation :

On a créé un projet **ESP32 sur Wokwi** et ajouté la librairie **PubSubClient**.
Le programme fait connecter l'ESP 32 au Wi-Fi **Wokwi-GUEST**, puis au broker MQTT public **test.mosquitto.org** sur le **port 1883** (sans SSL). Ensuite, l'ESP 32 publie régulièrement un message sur le topic **geiiiutv/syrine/n1**.

Pour vérifier la réception, on a configuré l'application **MyMQTT** sur le téléphone avec comme hôte : test.mosquitto.org, port : **1883**, sans identifiants et SSL désactivé, puis on s'est abonné au topic **geiiiutv/syrine/n1**. Les messages envoyés par l'ESP 32 s'affichent bien sur le téléphone, ce qui valide la communication MQTT.

Résultats :

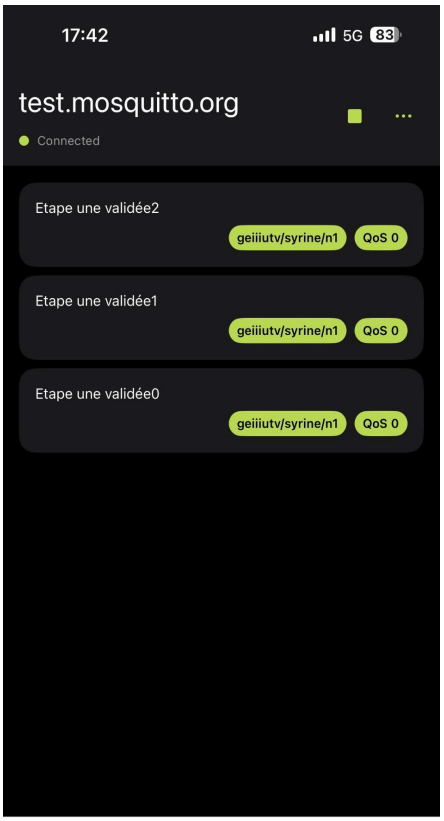
CODE en ANNEXE

Côté publisher :

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Connexion WiFi...
Connexion WiFi...
Connexion WiFi...
WiFi OK, IP=10.10.0.2
Connexion MQTT...
MQTT OK
Publish: Etape une validée0
```

On voit que la **connexion Wi-Fi est réussie** (WiFi OK, IP=10.10.0.2), ensuite la **connexion au broker MQTT** (MQTT OK). Enfin, l'ESP32 **publie un message** (Publish: Etape une validee0), ce qui confirme que l'envoi MQTT fonctionne.

<u>Côté subscriber :</u>



L'application est **connectée à au broker test.mosquitto.org** (statut Connected) et affiche les messages reçus sur le topic **geiiiutv/syrine/n1** : Étape une validée 0 ... Cela valide que les publications de l'ESP 32 sont bien **reçues côté client**.

<u>Conclusion de l'étape 1 :</u>

Cette première étape a permis de valider la communication MQTT de base. L'ESP 32 sur Wokwi se connecte au Wi-Fi puis au broker test.mosquitto.org sur le port **1883**, et publie des messages sur le topic geiiiutv/syrine/n1. Ces messages sont bien reçus par le client **MyMQTT**, ce qui confirme le bon fonctionnement publisher / subscriber. Mais cette configuration reste **non sécurisée** : aucun identifiant n'est utilisé et les échanges ne sont pas chiffrés.

2.2 Port 1884 - Authentification (login/password)

But : contrôler l'accès au broker par des identifiants.
Configuration : host test.mosquitto.org, port 1884, identifiants fournis par le serveur (rw / readwrite), topic geiiiutv/syrine/n2.
Sécurité : pas de TLS donc les données sont encore en clair.

Réalisation :

On a repris le projet **ESP32 sur Wokwi** et gardé la librairie **PubSubClient**.
Le programme fait connecter l'ESP 32 au Wi-Fi Wokwi-GUEST, puis au broker MQTT test.mosquitto.org sur le **port 1884** (sans SSL), en ajoutant une **authentification** (nom d'utilisateur + mot de passe). Ensuite, l'ESP 32 publie régulièrement un message sur le topic **geiiiutv/syrine/n2**.

Pour vérifier la réception, on a configuré l'application **MyMQTT** sur le téléphone avec comme hôte : test.mosquitto.org, port : **1884**, **SSL désactivé**, et les identifiants **(rw / readwrite)**, puis on s'est abonné au topic **geiiiutv/syrine/n2**. Les messages envoyés par l'ESP 32 s'affichent bien sur le téléphone, ce qui valide la communication MQTT avec authentification.

Résultats :

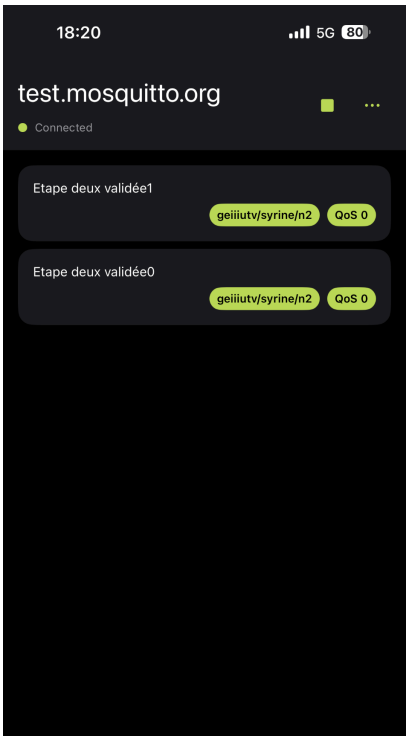
CODE en ANNEXE

Côté publisher :

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Connexion WiFi...
Connexion WiFi...
Connexion WiFi...
WiFi OK, IP=10.10.0.2
Connexion MQTT...
MQTT OK
Publish: Etape deux validée0
```

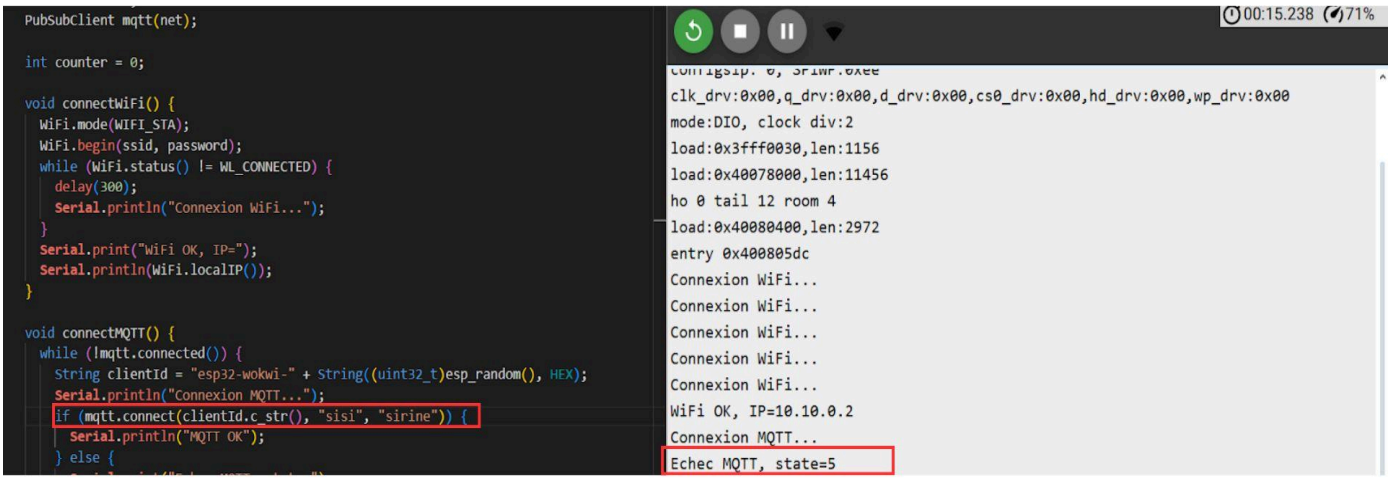
On voit l'ESP 32 démarrer puis se connecter au Wi-Fi (WiFi OK, IP=10.10.0.2). Ensuite, la connexion au broker MQTT est réussie (MQTT OK) et l'ESP 32 **publie** un message : Publish: Etape deux validée0 sur le topic de l'étape 2.

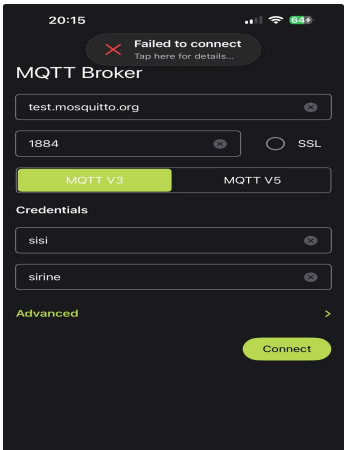
<u>Côté subscriber :</u>



L'application est **connectée** au broker test.mosquitto.org et affiche les messages reçus sur le topic **geiiiutv/syrine/n2** : Etape deux validée0 ... Cela confirme que les publications de l'ESP 32 sont bien **reçues** par le subscriber.

<u>Exemple avec un login et password mauvais:</u>





Conclusion de l'étape 2 :

Cette étape a permis de valider la communication MQTT avec **authentification**.
L'ESP32 sur Wokwi se connecte au broker test.mosquitto.org sur le port **1884** et publie des messages sur le topic geiiiutv/syrine/n2. Les messages sont bien reçus sur **MyMQTT**, mais uniquement si les **bons identifiants** sont renseignés, ce qui ajoute un contrôle d'accès. En revanche, la connexion reste **non chiffrée** : les données circulent encore en clair.

Sur le broker public test.mosquitto.org, les identifiants ne sont pas libres : il faut utiliser ceux fournis (ex : rw/readwrite).

2.3 Port 8883 - TLS (chiffrement + vérification serveur)

But : chiffrer la communication MQTT (TLS) et vérifier l'identité du serveur grâce au certificat CA (Certificate Authority).
Configuration : ESP32 en **TLS** (port **8883**) avec **WiFiClientSecure** + **CA mosquitto.org.crt** (+ heure **NTP**). Côté client, **MyMQTT en SSL** (port **8886**) sur le topic geiiiutv/syrine/n3.
Preuve attendue : log « MQTT OK » + réception côté client (mosquitto_sub en 8883 ou MyMQTT TLS).

Réalisation :

On a repris le projet ESP32 sur Wokwi et gardé la librairie **PubSubClient**, mais cette fois on a sécurisé l'échange avec **TLS**.

Côté ESP32, on a : remplacé WiFiClient par **WiFiClientSecure** pour activer TLS ;
ajouté dans le code le certificat CA **mosquitto.org.crt** (fourni par le broker) afin que l'ESP 32 puisse vérifier l'identité du serveur ;
mis l'ESP 32 à l'heure via **NTP**, nécessaire pour valider la période de validité du certificat ;
gardé la connexion au Wi-Fi Wokwi-GUEST, puis établi la connexion MQTT vers test.mosquitto.org en port 8883 (**MQTT chiffré**).

Une fois connecté, l'ESP 32 publie régulièrement des messages sur le topic geiiiutv/syrine/n3.

Côté téléphone, on a configuré MyMQTT avec SSL activé et on s'est abonné au topic **geiiiutv/syrine/n3** pour vérifier la réception. Dans notre cas, la connexion TLS sur le téléphone a été faite via le port 8886 (certificat Let's Encrypt, accepté par iOS contrairement au 8884), tout en recevant les messages du même broker.

Résultats :

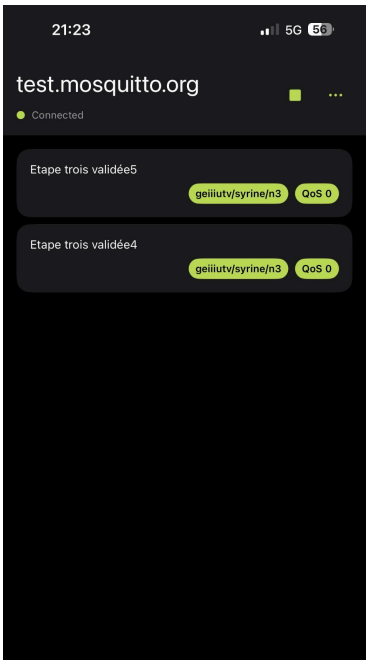
CODE en ANNEXE

Côté publisher :

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Connexion WiFi...
Connexion WiFi...
Connexion WiFi...
Connexion WiFi...
WiFi OK, IP=10.10.0.2
Connexion MQTT...
MQTT OK
Publish: Etape trois validée0
```

L'ESP32 se connecte au Wi-Fi puis établit une connexion MQTT chiffrée (TLS) vers test.mosquitto.org. Une fois la connexion réussie (MQTT OK), il publie régulièrement des messages sur le topic geiiiutv/syrine/n3 (Etape trois validée).

Côté subscriber :



L'application MyMQTT se connecte au broker test.mosquitto.org avec **SSL activé** (TLS), puis s'abonne au topic geiiiutv/syrine/n3. Les messages publiés par l'ESP 32 apparaissent dans l'application, ce qui confirme la **réception** côté subscriber

Conclusion de l'étape 3 :

Cette étape a permis de valider une communication MQTT **sécurisée par TLS**. L'ESP32 (sur Wokwi) se connecte au broker test.mosquitto.org en mode chiffré et publie correctement des messages sur le topic geiiiutv/syrine/n3, reçus côté client. Grâce au chiffrement, les données **ne circulent plus en clair** : la communication est protégée contre l'écoute et la modification sur le réseau. De plus, l'utilisation du **certificat CA** permet de vérifier l'identité du serveur, même sans authentification par mot de passe.

2.4 Port 8884 - mTLS (certificat client)

But : authentifier le client par certificat (mTLS).
Configuration : port 8884, ajout dans l'ESP 32 : CA + certificat client + clé privée.
Preuve forte : mosquitto_sub (WSL) reçoit le message avec --cafile/--cert/--key

Réalisation :

L'objectif ici n'est plus seulement de **chiffrer** la communication (comme en 8883), mais aussi de **prouver l'identité du client** (l'ESP 32) au serveur grâce à un **certificat client**. On appelle ça du **mTLS** (mutual TLS).

1) Création des "preuves d'identité" du client (sur PC sous Windows)

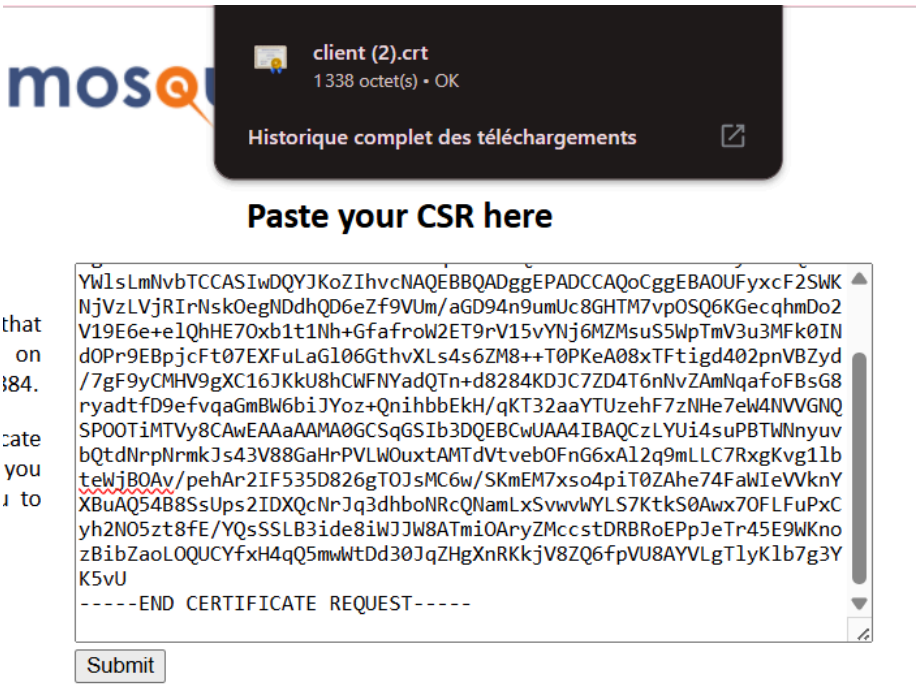
```
MINGW64:/c/Users/allou/mqtt-cert

allou@Sirinee MINGW64 ~
$ mkdir mqtt-cert cd mqtt-cert
mkdir: cannot create directory 'mqtt-cert': File exists
mkdir: cannot create directory 'mqtt-cert': File exists

allou@Sirinee MINGW64 ~
$ cd mqtt-cert

allou@Sirinee MINGW64 ~/mqtt-cert
$ ls -l
total 0

allou@Sirinee MINGW64 ~/mqtt-cert
$ openssl genrsa -out client.key 2048

allou@Sirinee MINGW64 ~/mqtt-cert
$ openssl req -new -key client.key -out client.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:Sirine
String too long, must be at most 2 bytes long
Country Name (2 letter code) [AU]:France
String too long, must be at most 2 bytes long
Country Name (2 letter code) [AU]:Fr
State or Province Name (full name) [Some-State]:93
Locality Name (eg, city) []:Villetaneuse
Organization Name (eg, company) [Internet Widgits Pty Ltd]:iut
Organizational Unit Name (eg, section) []:geii
Common Name (e.g. server FQDN or YOUR name) []:syrine_n4
Email Address []:alloutsirine@gmail.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:

allou@Sirinee MINGW64 ~/mqtt-cert
$ ls -l
total 8
-rw-r--r-- 1 allou 197609 1068 Jan  5 11:20 client.csr
-rw-r--r-- 1 allou 197609 1732 Jan  5 11:17 client.key

allou@Sirinee MINGW64 ~/mqtt-cert
$ cat client.csr
-----BEGIN CERTIFICATE REQUEST-----
MIICzzCCAbcCAQAwgYkxCzAJBgNVBAYTAkZyMQswCQYDVQQIDAI5MzEVMBMGA1UE
BwwMVmlsbGV0YW5ldXNlMQwwCgYDVQQKDANpdXQxDTALBgNVBAsMBGdlaWkxEjAQ
BgNVBAMMCXN5cmluZV9uNDElMCMGCSqGSIb3DQEJARYWYwxsb3V0c2lyaW5lQGdt
YWlsLmNvbTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAOUFyxcF2SWK
NjVzLVjRIrNskOegNDdhQD6eZf9VUm/aGD94n9umUc8GHTM7vpOSQ6KGecqhmDo2
V19E6e+elQhHE7Oxb1t1Nh+GfafroW2ET9rV15vYNj6MZMsuS5WpTmV3u3MFkOIN
```

```
allou@Sirinee MINGW64 ~/mqtt-cert
$ ls -l
total 8
-rw-r--r-- 1 allou 197609 1068 Jan  5 11:20 client.csr
-rw-r--r-- 1 allou 197609 1732 Jan  5 11:17 client.key

allou@Sirinee MINGW64 ~/mqtt-cert
$ cat client.csr
-----BEGIN CERTIFICATE REQUEST-----
MIICzzCCAbcCAQAwgYkxCzAJBgNVBAYTAkZyMQswCQYDVQQIDAI5MzEVMBMGA1UE
BwwMVmlsbGVOYW5ldXNlMQwwCgYDVQQKDANpdXQxDTALBgNVBAsMBGdlaWkxEjAQ
BgNVBAMMCXN5cmluZV9uNDElMCMGCSqGSIb3DQEJARYWYWxsb3VOc2lyaW5lQGdt
YWlsLmNvbTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAOUFyxcF2SWK
NjVzLVjRIrNskOegNDdhQD6eZf9VUm/aGD94n9umUc8GHTM7vpOSQ6KGecqhmDo2
V19E6e+elQhHE7Oxb1t1Nh+GfafroW2ET9rV15vYNj6MZMsuS5WpTmV3u3MFkOIN
dOPr9EBpjcFt07EXFuLaGl06GthvXLs4s6ZM8++T0PKeA08xTFtigd402pnVBZyd
/7gF9yCMHV9gXC16JKkU8hCWFNYadQTn+d8284KDJC7ZD4T6nNvZAmNqafoFBsG8
ryadtfD9efvqaGmBW6biJYoz+QnihbbEkH/qKT32aaYTUzehF7zNHe7eW4NVVGNQ
SPOOTiMTVy8CAwEAAaAAMA0GCSqGSIb3DQEBCwUAA4IBAQCzLYUi4suPBTWNnyuv
bQtdNrpNrmkJs43V88GaHrPVLWOuxtAMTdVtvebOFnG6xAl2q9mLLC7RxgKvg1lb
teWjBOAv/pehAr2IF535D826gTOJsMC6w/SKmEM7xso4piT0ZAhe74FaWIeVVknY
XBuAQ54B8SsUps2IDXQcNrJq3dhboNRcQNamLxSvwvWYLS7KtkSOAwx7OFLFuPxC
yh2NO5zt8fE/YQsSSLB3ide8iWJJW8ATmiOAryZMccstDRBRoEPpJeTr45E9WKno
zBibZaoLOQUCYfxH4qQ5mwWtDd30JqZHgXnRKkjV8ZQ6fpVU8AYVLgTlyKlb7g3Y
K5vU
-----END CERTIFICATE REQUEST-----

allou@Sirinee MINGW64 ~/mqtt-cert
$ |
```

On a d'abord généré une **clé privée** pour le client (client.key).
Ensuite, on a créé une **demande de certificat** (client.csr) : c'est un fichier qui contient les infos d'identité (nom, organisation, etc.) et qui sert à demander un certificat signé.

Comme on est sous **Windows**, la création des fichiers se fait dans un terminal **Git Bash / MINGW64** (les commandes ne sont pas exactement les mêmes que sur Linux).
On a généré : **client.key** : **clé privée** du client (à garder secrète) ; **client.csr** : **la demande de certificat** (elle contient les infos d'identité que l'on rentre dans le terminal et qui servent à obtenir un certificat sur Mosquitto.org)

2) Récupération du certificat client



À partir du client.csr, on obtient ensuite un **certificat client signé** client.crt (via l'option "generate your own certificate" sur Mosquitto).

On a donc le trio nécessaire au mTLS :
client.crt (certificat client) ; **client.key** (clé privée) et **mosquitto.org.crt** (certificat **CA** pour vérifier le serveur).

(le client.csr (généré par client.key), sert à obtenir le client.crt.)
en gros client.key -> client.csr -> client.crt (c'est lui qui permet au broker de reconnaître "ce client est autorisé")

4) Modification du programme ESP32 (Wokwi)

Dans le code, on a : remplacé WiFiClient par **WiFiClientSecure** (indispensable pour TLS) ; ajouté la **mise à l'heure NTP** (sinon le certificat peut être refusé car "pas à la bonne date") ; chargé les certificats dans l'ESP 32 : **setCACert(MOSQUITTO_CA)** qui vérifie le serveur et **setCertificate(CLIENT_CRT)** + **setPrivateKey(CLIENT_KEY)** qui prouvent l'identité de l'ESP32.
Enfin, on se connecte au broker test.mosquitto.org sur le **port 8884** (mode mTLS), puis on publie régulièrement sur le topic : **geiiiutv/syrine/n4 .**

CODE en ANNEXE

Résultat sur Wokwi:

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Connexion WiFi...
Connexion WiFi...
Connexion WiFi...
WiFi OK, IP=10.10.0.2
Mise a l'heure NTP............. OK
Connexion MQTT (mTLS 8884)...
MQTT OK
Publish: Etape 4 validée0
Publish: Etape 4 validée1
Publish: Etape 4 validée2
```

5) Vérification côté subscriber

```
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$ wget --no-check-certificate -O mosquitto.org.crt https
://test.mosquitto.org/ssl/mosquitto.org.crt
--2026-01-08 22:20:45--  https://test.mosquitto.org/ssl/mosquitto.org.crt
Resolving test.mosquitto.org (test.mosquitto.org)... 54.36.178.49, 2001:41d0:303:4831::1
Connecting to test.mosquitto.org (test.mosquitto.org)|54.36.178.49|:443... connected.
WARNING: cannot verify test.mosquitto.org's certificate, issued by 'CN=R12,O=Let's Encrypt,C=US':
  Issued certificate has expired.
HTTP request sent, awaiting response... 200 OK
Length: 1452 (1.4K) [application/x-x509-ca-cert]
Saving to: 'mosquitto.org.crt'

mosquitto.org.crt   100%[===============>]   1.42K  --.-KB/s    in 0s

2026-01-08 22:20:45 (77.0 MB/s) - 'mosquitto.org.crt' saved [1452/1452]

allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$ ls -l
total 12
-rwxrwxrwx 1 allout allout 1068 Jan  5 11:20 client.csr
-rwxrwxrwx 1 allout allout 1732 Jan  5 11:17 client.key
-rwxrwxrwx 1 allout allout 1452 Jun  9  2020 mosquitto.org.crt
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$
```

Pour vérifier la connexion **mTLS en 8884**, on a utilisé mosquitto_sub sur **WSL**.
Avant de lancer le test, on a téléchargé le certificat CA du broker (mosquitto.org.crt) depuis le site de Mosquitto (commande wget), puis on a vérifié la présence des fichiers client.csr, client.key et mosquitto.org.crt dans le dossier.

```
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$ cd /mnt/c/Users/allou/mqtt-cert && mosquitto_sub -h test.mosquitto.org
 -p 8884 -t "geiiiutv/syrine/n4" -v --cafile mosquitto.org.crt --cert client.crt --key client.key
geiiiutv/syrine/n4 N4 OK - TLS + auth (websocket)
geiiiutv/syrine/n4 N4 OK - TLS + auth (websocket)
geiiiutv/syrine/n4 N4 OK - TLS + auth (websocket)
```

Ensuite, on s'est abonné au topic geiiiutv/syrine/n4 avec **mosquitto_sub** en fournissant les trois fichiers nécessaires : **--cafile mosquitto.org.crt, --cert client.crt et --key client.key**.
Quand tout est correct, la connexion est acceptée et les messages publiés apparaissent côté subscriber, ce qui valide l'étape.

```
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$ cd /mnt/c/Users/allou/mqtt-cert && mosquitto_sub -h test.mosquitto.org
 -p 8884 -t "geiiiutv/syrine/n4" -v --cafile mosquitto.org.crt --cert client.crt --key client.key
geiiiutv/syrine/n4 N4 OK - TLS + auth (websocket)
geiiiutv/syrine/n4 N4 OK - TLS + auth (websocket)
geiiiutv/syrine/n4 N4 OK - TLS + auth (websocket)
geiiiutv/syrine/n4 N4 OK - TLS + auth (websocket)
geiiiutv/syrine/n4 N4 OK - TLS + auth (websocket)
geiiiutv/syrine/n4 N4 OK - TLS + auth (websocket)
geiiiutv/syrine/n4 N4 OK - TLS + auth (websocket)
geiiiutv/syrine/n4 TEST SUBSCRIBER 8884 OK
```

```
allout@Sirinee: /mnt/c/Users/allou/mqtt-cert
allout@Sirinee:/mnt/c/WINDOWS/system32$ cd /mnt/c/Users/allou/mqtt-cert
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$ ls -l
total 12
-rwxrwxrwx 1 allout allout    0 Jan  8 22:41 client.crt.save
-rwxrwxrwx 1 allout allout 1068 Jan  5 11:20 client.csr
-rwxrwxrwx 1 allout allout 1732 Jan  5 11:17 client.key
-rwxrwxrwx 1 allout allout 1452 Jun  9  2020 mosquitto.org.crt
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$ ls -l client.crt
-rwxrwxrwx 1 allout allout 1358 Jan  9 00:00 client.crt
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$ head -n 2 client.crt
-----BEGIN CERTIFICATE-----
MIIDsDCCApigAwIBAgIBADANBgkqhkiG9w0BAQsFADCBkDELMAkGA1UEBhMCR0Ix
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$ tail -n 2 client.crt
g0xRoG6FrobGqOyDjW76cadjHN0c3q5eKqnJYu/b4oG6H+UB
-----END CERTIFICATE-----allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$aaaaaaaaaaaaaaaaallout@Sirinee:/mntallout@Siri
nee:/mnt/c/Users/allou/mqtt-callalloutallout@Sirinallout@Sirinee:/mnt/c/Usersallout@Sirinallout@Sirinee:/mnt/c/Usal
lout@Sirinee:allout@Sirinee:/mnalloutallout@Sirinee:/mnt/calloutallout@Siallalloutallout@Sirinee:/mnt/c/Users/allou
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$ cd /mnt/c/Users/allou/mqtt-cert && mosquitto_pub -h test.mosquitto.org
 -p 8884 -t "geiiiutv/syrine/n4" -m "TEST SUBSCRIBER 8884 OK" --cafile mosquitto.org.crt --cert client.crt --key cl
ient.key
allout@Sirinee:/mnt/c/Users/allou/mqtt-cert$
```

Sur cette capture, on valide le **mTLS sur le port 8884** en montrant à la fois la **réception** et l'**envoi** de messages, avec les fichiers de certificats.

Fenêtre du bas : on vérifie d'abord que le fichier **client.crt** est bien présent (ls -l client.crt), puis on affiche rapidement son contenu avec head et tail pour confirmer que c'est bien un certificat (BEGIN CERTIFICATE et END CERTIFICATE).
Ensuite, on utilise mosquitto_pub en 8884 (avec --cafile, --cert, --key) pour envoyer le message de test **"TEST SUBSCRIBER 8884 OK"**. (envoie)

Fenêtre du haut : on lance mosquitto_sub en 8884 sur le topic geiiiutv/syrine/n4 avec --cafile mosquitto.org.crt, --cert client.crt et --key client.key. Les lignes **"N4 OK - TLS + auth"** confirment que la connexion est bien établie en **TLS** et que l'accès est accepté grâce au **certificat client**. (déjà vu dans le screen juste avant celui là)

! La ligne **"TEST SUBSCRIBER 8884 OK"** montre qu'un message a bien été reçu côté subscriber ! (réception)

2.5 Port 8885 :

```
Connexion WiFi...
Connexion WiFi...
Connexion WiFi...
Connexion WiFi...
Connexion WiFi...
Connexion WiFi...
Connexion WiFi...
WiFi OK, IP=192.168.1.65
Mise a l'heure NTP............... OK
Connexion MQTT (TLS+AUTH 8885)...
Echec MQTT, state=5
Connexion MQTT (TLS+AUTH 8885)...
Echec MQTT, state=5
Connexion MQTT (TLS+AUTH 8885)...
Echec MQTT, state=5
Connexion MQTT (TLS+AUTH 8885)...
Echec MQTT, state=5
```

ANNEXE :

Codes utilisés dans les étapes n1; n2; n3; n4 :

CODE 1883 :

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "Wokwi-GUEST";
const char* password = "";

const char* mqttHost = "test.mosquitto.org";
const int mqttPort = 1883;  // Niveau 1
const char* topic = "geiiiutv/syrine/n1";

WiFiClient net;
PubSubClient mqtt(net);

int counter = 0;

void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(300);
    Serial.println("Connexion WiFi...");
  }
  Serial.print("WiFi OK, IP=");
  Serial.println(WiFi.localIP());
}

void connectMQTT() {
  while (!mqtt.connected()) {
```

```
    String clientId = "esp32-wokwi-" + String((uint32_t)esp_random(), HEX);
    Serial.println("Connexion MQTT...");
    if (mqtt.connect(clientId.c_str())) {
      Serial.println("MQTT OK");
    } else {
      Serial.print("Echec MQTT, state=");
      Serial.println(mqtt.state());
      delay(1500);
    }
  }
}

void setup() {
  Serial.begin(115200);
  connectWiFi();
  mqtt.setServer(mqttHost, mqttPort);
}

void loop() {
  if (!mqtt.connected()) connectMQTT();
  mqtt.loop();

  static unsigned long last = 0;
  if (millis() - last > 2000) {
    last = millis();
    String msg = "Etape une validée" + String(counter++);
    mqtt.publish(topic, msg.c_str());
    Serial.println("Publish: " + msg);
  }
}
```

CODE 1884 :

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "Wokwi-GUEST";
const char* password = "";

const char* mqttHost = "test.mosquitto.org";
const int mqttPort = 1884;  // Niveau 2
const char* topic = "geiiiutv/syrine/n2";

WiFiClient net;
PubSubClient mqtt(net);

int counter = 0;

void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(300);
    Serial.println("Connexion WiFi...");
  }
  Serial.print("WiFi OK, IP=");
  Serial.println(WiFi.localIP());
}

void connectMQTT() {
  while (!mqtt.connected()) {
    String clientId = "esp32-wokwi-" + String((uint32_t)esp_random(), HEX);
    Serial.println("Connexion MQTT...");
    if (mqtt.connect(clientId.c_str(), "rw", "readwrite")) {
      Serial.println("MQTT OK");
```

```
    } else {
      Serial.print("Echec MQTT, state=");
      Serial.println(mqtt.state());
      delay(1500);
    }
  }
}

void setup() {
  Serial.begin(115200);
  connectWiFi();
  mqtt.setServer(mqttHost, mqttPort);
}

void loop() {
  if (!mqtt.connected()) connectMQTT();
  mqtt.loop();

  static unsigned long last = 0;
  if (millis() - last > 2000) {
    last = millis();
    String msg = "Etape deux validée" + String(counter++);
    mqtt.publish(topic, msg.c_str());
    Serial.println("Publish: " + msg);
  }
}
```

CODE 8883 :

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <WiFiClientSecure.h>

static const char* MOSQUITTO_CA PROGMEM = R"EOF(
-----BEGIN CERTIFICATE-----
MIIEAzCCAuugAwIBAgIUBY1hlCGvdj4NhBXkZ/uLUZNILAwwDQYJKoZIhvcNAQEL
BQAwgZAxCzAJBgNVBAYTAkdCMRcwFQYDVQQIDA5Vbml0ZWQgS2luZ2RvbTEOMAwG
A1UEBwwFRGVyYnkxEjAQBgNVBAoMCU1vc3F1aXR0bzELMAkGA1UECwwCQ0ExFjAU
BgNVBAMMDW1vc3F1aXR0by5vcmcxHzAdBgkqhkiG9w0BCQEWEHJvZ2VyQGF0Y2hv
by5vcmcwHhcNMjAwNjA5MTEwNjM5WhcNMzAwNjA3MTEwNjM5WjCBkDELMAkGA1UE
BhMCR0IxFzAVBgNVBAgMDlVuaXRlZCBLaW5nZG9tMQ4wDAYDVQQHDAVEZXJieTES
MBAGA1UECgwJTW9zcXVpdHRvMQswCQYDVQQLDAJDQTEWMBQGA1UEAwwNbW9zcXVp
dHRvLm9yZzEfMB0GCSqGSIb3DQEJARYQcm9nZXJAYXRjaG9vLm9yZzCCASIwDQYJ
KoZIhvcNAQEBBQADggEPADCCAQoCggEBAME0HKmIzfTOwkKLT3THHe+ObdizamPg
UZmD64Tf3zJdNeYGYn4CEXbyP6fy3tWc8S2boW6dzrH8SdFf9uo320GJA9B7U1FW
Te3xda/Lm3JFfaHjkWw7jBwcauQZjpGINHapHRlpiCZsquAthOgxW9SgDgYlGzEA
s06pkEFiMw+qDfLo/sxFKB6vQlFekMeCymjLCbNwPJyqyhFmPWwio/PDMruBTzPH
3cioBnrJWKXc3OjXdLGFJOfj7pP0j/dr2LH72eSvv3PQQFl90CZPFhrCUcRHSSxo
E6yjGOdnz7f6PveLIB574kQORwt8ePn0yidrTC1ictikED3nHYhMUOUCAwEAAaNT
MFEwHQYDVR0OBBYEFPVV6xBUFPiGKDyo5V3+Hbh4N9YSMB8GA1UdIwQYMBaAFPVV
6xBUFPiGKDyo5V3+Hbh4N9YSMA8GA1UdEwEB/wQFMAMBAf8wDQYJKoZIhvcNAQEL
BQADggEBAGa9kS21N70ThM6/Hj9D7mbVxKLBjVWe2TPsGfbl3rEDfZ+OKRZ2j6AC
6r7jb4TZO3dzF2p6dgbrlU71Y/4K0TdzIjRj3cQ3KSm41JvUQ0hZ/c04iGDg/xWf
+pp58nfPAYwuerruPNWmlStWAXf0UTqRtg4hQDWBuUFDJTuWuuBvEXudz74eh/wK
sMwfu1HFvjy5Z0iMDU8PUDepjVolOCue9ashlS4EB5IECdSR2TltnAIiIwimx839
LdUdRudafMu5T5Xma182OC0/u/xRlEm+tvKGGmfFcN0piqVl8OrSPBgIIb+1IKJE
m/XriWr/Cq4h/JfB7NTsezVslgkBaoU=
-----END CERTIFICATE-----
)EOF";
```

```
const char* ssid = "Wokwi-GUEST";
const char* password = "";

const char* mqttHost = "test.mosquitto.org";
const int mqttPort = 8883;  // Niveau 1
const char* topic = "geiiiutv/syrine/n3";  // remplace <TON_PRENOM>



WiFiClientSecure net;
PubSubClient mqtt(net);



int counter = 0;

void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(300);
    Serial.println("Connexion WiFi...");
  }
  Serial.print("WiFi OK, IP=");
  Serial.println(WiFi.localIP());
}

void connectMQTT() {
  while (!mqtt.connected()) {
    String clientId = "esp32-wokwi-" + String((uint32_t)esp_random(), HEX);
    Serial.println("Connexion MQTT...");
     if (mqtt.connect(clientId.c_str())) {
      Serial.println("MQTT OK");
    } else {
      Serial.print("Echec MQTT, state=");
```

```
      Serial.println(mqtt.state());
      delay(1500);
    }
  }
}

void setup() {
  Serial.begin(115200);
  connectWiFi();
  net.setCACert(MOSQUITTO_CA);
  mqtt.setServer(mqttHost, mqttPort);
}

void loop() {
  if (!mqtt.connected()) connectMQTT();
  mqtt.loop();

  static unsigned long last = 0;
  if (millis() - last > 2000) {
    last = millis();
    String msg = "Etape trois validée" + String(counter++);
    mqtt.publish(topic, msg.c_str());
    Serial.println("Publish: " + msg);
  }
}
```

CODE 8884 :

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <time.h>

// 1) CA (mosquitto.org.crt)
static const char* MOSQUITTO_CA PROGMEM = R"EOF(
-----BEGIN CERTIFICATE-----
MIIEAzCCAuugAwIBAgIUBY1hlCGvdj4NhBXkZ/uLUZNILAwwDQYJKoZIhvcNAQEL
BQAwgZAxCzAJBgNVBAYTAkdCMRcwFQYDVQQIDA5Vbml0ZWQgS2luZ2RvbTEOMAwG
A1UEBwwFRGVyYnkxEjAQBgNVBAoMCU1vc3F1aXR0bzELMAkGA1UECwwCQ0ExFjAU
BgNVBAMMDW1vc3F1aXR0by5vcmcxHzAdBgkqhkiG9w0BCQEWEHJvZ2VyQGF0Y2hv
by5vcmcwHhcNMjAwNjA5MTEwNjM5WhcNMzAwNjA3MTEwNjM5WjCBkDELMAkGA1UE
BhMCR0IxFzAVBgNVBAgMDlVuaXRlZCBLaW5nZG9tMQ4wDAYDVQQHDAVEZXJieTES
MBAGA1UECgwJTW9zcXVpdHRvMQswCQYDVQQLDAJDQTEWMBQGA1UEAwwNbW9zcXVp
dHRvLm9yZzEfMB0GCSqGSIb3DQEJARYQcm9nZXJAYXRjaG9vLm9yZzCCASIwDQYJ
KoZIhvcNAQEBBQADggEPADCCAQoCggEBAME0HKmIzfTOwkKLT3THHe+ObdizamPg
UZmD64Tf3zJdNeYGYn4CEXbyP6fy3tWc8S2boW6dzrH8SdFf9uo320GJA9B7U1FW
Te3xda/Lm3JFfaHjkWw7jBwcauQZjpGINHapHRlpiCZsquAthOgxW9SgDgYlGzEA
s06pkEFiMw+qDfLo/sxFKB6vQlFekMeCymjLCbNwPJyqyhFmPWwio/PDMruBTzPH
3cioBnrJWKXc3OjXdLGFJOfj7pP0j/dr2LH72eSvv3PQQFl90CZPFhrCUcRHSSxo
E6yjGOdnz7f6PveLIB574kQORwt8ePn0yidrTC1ictikED3nHYhMUOUCAwEAAaNT
MFEwHQYDVR0OBBYEFPVV6xBUFPiGKDyo5V3+Hbh4N9YSMB8GA1UdIwQYMBaAFPVV
6xBUFPiGKDyo5V3+Hbh4N9YSMA8GA1UdEwEB/wQFMAMBAf8wDQYJKoZIhvcNAQEL
BQADggEBAGa9kS21N70ThM6/Hj9D7mbVxKLBjVWe2TPsGfbl3rEDfZ+OKRZ2j6AC
6r7jb4TZO3dzF2p6dgbrlU71Y/4K0TdzIjRj3cQ3KSm41JvUQ0hZ/c04iGDg/xWf
+pp58nfPAYwuerruPNWmlStWAXf0UTqRtg4hQDWBuUFDJTuWuuBvEXudz74eh/wK
sMwfu1HFvjy5Z0iMDU8PUDepjVolOCue9ashlS4EB5IECdSR2TltnAIiIwimx839
LdUdRudafMu5T5Xma182OC0/u/xRlEm+tvKGGmfFcN0piqVl8OrSPBgIIb+1IKJE
m/XriWr/Cq4h/JfB7NTsezVslgkBaoU=
-----END CERTIFICATE-----
```

```
)EOF";

// 2) Certificat client (client.crt)
static const char* CLIENT_CRT PROGMEM = R"EOF(
-----BEGIN CERTIFICATE-----
MIIDsDCCApigAwIBAgIBADANBgkqhkiG9w0BAQsFADCBkDELMAkGA1UEBhMCR0Ix
FzAVBgNVBAgMDlVuaXRlZCBLaW5nZG9tMQ4wDAYDVQQHDAVEZXJieTESMBAGA1UE
CgwJTW9zcXVpdHRvMQswCQYDVQQLDAJDQTEWMBQGA1UEAwwNbW9zcXVpdHRvLm9y
ZzEfMB0GCSqGSIb3DQEJARYQcm9nZXJAYXRjaG9vLm9yZzAeFw0yNjAxMDUxMDQ2
NDFaFw0yNjA0MDUxMDQ2NDFaMIGJMQswCQYDVQQGEwJGcjELMAkGA1UECAwCOTMx
FTATBgNVBAcMDFZpbGxldGFuZXVzZTEMMAoGA1UECgwDaXV0MQ0wCwYDVQQLDARn
ZWlpMRIwEAYDVQQDDAlzeXJpbmVfbjQxJTAjBgkqhkiG9w0BCQEWFmFsbG91dHNp
cmluZUBnbWFpbC5jb20wggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQDl
BcsXBdklijY1cy1Y0SKzbJDnoDQ3YUA+nmX/VVJv2hg/eJ/bplHPBh0zO76TkkOi
hnnKoZg6NldfROnvnpUIRxOzsW9bdTYfhn2n66FthE/a1deb2DY+jGTLLkuVqU5l
d7tzBZNCDXTj6/RAaY3BbdOxFxbi2hpdOhrYb1y7OLOmTPPvk9DyngNPMUxbYoHe
NNqZ1QWcnf+4BfcgjB1fYFwteiSpFPIQlhTWGnUE5/nfNvOCgyQu2Q+E+pzb2QJj
amn6BQbBvK8mnbXw/Xn76mhpgVum4iWKM/kJ4oW2xJB/6ik99mmmE1M3oRe8zR3u
3luDVVRjUEjzjk4jE1cvAgMBAAGjGjAYMAkGA1UdEwQCMAAwCwYDVR0PBAQDAgXg
MA0GCSqGSIb3DQEBCwUAA4IBAQCsx6WOpXdCr8Y/dBlcwo3qRU+O66niOO/CftHQ
mz4laLitkT0S3q8D75F2FBr5Fnlqp5kWkPBYutw1F7kQ93Z40WpbxXVBRgl5yVrn
ijk2uLM8H8yauTgnq8mcedwwg2RIxAFI7Cfr0VK4fQ/9VXE2hIU2CWTMGy8T7KAN
B9dK978EwZsZ+qDM2JrTvCrWkaZDplm2mRtb2BegK9l466aS0P5dZyOiWtdScRmQ
IGvdbOmVO0SDpWT38gRZOY6kEpFofewpMZN8fKcG81ZJ0qmUgI1EBU5i0AVmGOWc
g0xRoG6FrobGqOyDjW76cadjHN0c3q5eKqnJYu/b4oG6H+OB
-----END CERTIFICATE-----
)EOF";

// 3) Clé privée (client.key)
static const char* CLIENT_KEY PROGMEM = R"EOF(
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQDlBcsXBdklijY1
cy1Y0SKzbJDnoDQ3YUA+nmX/VVJv2hg/eJ/bplHPBh0zO76TkkOihnnKoZg6Nldf
```

ROnvnpUIRxOzsW9bdTYfhn2n66FthE/a1deb2DY+jGTLLkuVqU5ld7tzBZNCDXTj
6/RAaY3BbdOxFxbi2hpdOhrYb1y7OLOmTPPvk9DyngNPMUxbYoHeNNqZ1QWcnf+4
BfcgjB1fYFwteiSpFPIQlhTWGnUE5/nfNvOCgyQu2Q+E+pzb2QJjamn6BQbBvK8m
nbXw/Xn76mhpgVum4iWKM/kJ4oW2xJB/6ik99mmmE1M3oRe8zR3u3luDVVRjUEjz
jk4jE1cvAgMBAAECggEAQGVaHw2eap9LscjCcexQKQo4TmCt/fpi4ZiRTnOzu86j
rckhjdXcv+zUrhC4++6PBPZx7Wd1kbyw4ZFTg/RfrFWr9QVJrfotrzyWPW6UU+J1
v68MMn5fdkPsnXLVfu39pKs/ck6ZguOcv++bHLdxS/exI+S9oyi3bApH/R6HAP8e
qbLuzxrPSePB0N78Sycd2prvaWx5Jyitt45TJ47TBNUsaeYh7RSMR4+I9SNaGVi8
EPcLm/JKJQFprAN1nfgQcSCV5gij7Vm8n3QdCrSdFqRk74HYTnZczJpSvMKacr6A
gpjfdKLyNpXF2hu8gfscTHeIEa+ZM7zV+GPwwLlCsQKBgQD/sUyfs8XErpIbRVNz
LlOxJhw+vDr48NeT7vDGeRlKLlvDhNuRh7Y8qFpSb1dTZqy/K/dK25ussagDklF7
IV/syKwAHJVnpI+DU8mNlasdh2UCZqe5LOmkCBp5J1XKc9yCiuLl2nI79jxnuKU+
XGsALtWcqtsXWYtCuVbJgzP/XwKBgQDlTEj/27stPkmbKfCPF++XofYyixOFxKbM
BycklVVS2jdgSiwQJqY97pomdUoMw1VAMRqu91P21eENba6TYdStbHDBXSWv9yan
9JOmHn4BsX6apLZPJznYN6RpIpv36sudDZ6MqA/sSd0KUxxZN5nQPLEj6K/9PXnm
IEuMenJKMQKBgGilYZzC8J/sanOqzdqIDUylMk4ir2gO+cvbsMMp8MXBb2D30P0g
s97jcAPo+a5yiSS8k7sZYf8tvnunuJ1SLXY1Mt/g0uzWFr5C3myJ4rx1G0v+Q4qv
7PdrMZqAFMaE4Ky0tuQnoTUVM+enVEnPfSNYB6jU7FIEHdCzFefnQLK1AoGBAIjc
YJH+yBj4HZz0inYOVl6OOFHwuwVT3C7Zch6e15+tSfI6M1SsjPtYSD/+L+WwTVkU
NhYK393ryPln5xRnBzdAC4kOnOkLvtHu5JX5i0Jde/C3sW1LP3Zzt7ldA/UU+tVS
BLl1spNZfwqjkr8ZtcAaCa0iPLENmY2m5xEBIOhxAoGABv3x+vdrI3T1OtRA/uo0
JD+nl/U6T/+QPQqoZShz+s1MgIKOJRhtpp8BkuFIcxw8ezn2lJzFObDWhh5Y/6DO
cvyQwiI3GTPSyc7b2DuPJtmry27MiJLuzIa9Pb1VoT9jDIMjPidhXNA3MrHs2ys9
LlijsWqMxILoBxJy8IKPU+0=
-----END PRIVATE KEY-----
)EOF";

```
// WiFi
const char* ssid = "Wokwi-GUEST";
const char* password = "";

// MQTT
const char* mqttHost = "test.mosquitto.org";
```

```
const int   mqttPort = 8884;
const char* topic    = "geiiiutv/syrine/n4";

WiFiClientSecure net;
PubSubClient mqtt(net);

int counter = 0;

void setupTime() {
  configTime(0, 0, "pool.ntp.org", "time.nist.gov");
  Serial.print("Mise a l'heure NTP...");
  time_t now = time(nullptr);
  while (now < 1700000000) {  // attend une vraie date (sinon TLS échoue)
    delay(300);
    Serial.print(".");
    now = time(nullptr);
  }
  Serial.println(" OK");
}

void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(300);
    Serial.println("Connexion WiFi...");
  }
  Serial.print("WiFi OK, IP=");
  Serial.println(WiFi.localIP());
}

void connectMQTT() {
  while (!mqtt.connected()) {
```

```
    String clientId = "esp32-" + String((uint32_t)esp_random(), HEX);
    Serial.println("Connexion MQTT (mTLS 8884)...");
    if (mqtt.connect(clientId.c_str())) {
      Serial.println("MQTT OK");
    } else {
      Serial.print("Echec MQTT, state=");
      Serial.println(mqtt.state());
      delay(1500);
    }
  }
}

void setup() {
  Serial.begin(115200);
  connectWiFi();
  setupTime();

  net.setCACert(MOSQUITTO_CA);
  net.setCertificate(CLIENT_CRT);
  net.setPrivateKey(CLIENT_KEY);

  mqtt.setServer(mqttHost, mqttPort);
}

void loop() {
  if (!mqtt.connected()) connectMQTT();
  mqtt.loop();

  static unsigned long last = 0;
  if (millis() - last > 2000) {
    last = millis();
    String msg = "Etape 8884 mTLS OK " + String(counter++);
    mqtt.publish(topic, msg.c_str());
```

```
    Serial.println("Publish: " + msg);
  }
}
```

Certificats :

CLIENT.KEY : Clé privée du client

-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQDlBcsXBdklijY1
cy1Y0SKzbJDnoDQ3YUA+nmX/VVJv2hg/eJ/bplHPBh0zO76TkkOihnnKoZg6Nldf
ROnvnpUIRxOzsW9bdTYfhn2n66FthE/a1deb2DY+jGTLLkuVqU5ld7tzBZNCDXTj
6/RAaY3BbdOxFxbi2hpdOhrYb1y7OLOmTPPvk9DyngNPMUxbYoHeNNqZ1QWcnf+4
BfcgjB1fYFwteiSpFPIQlhTWGnUE5/nfNvOCgyQu2Q+E+pzb2QJjamn6BQbBvK8m
nbXw/Xn76mhpgVum4iWKM/kJ4oW2xJB/6ik99mmmE1M3oRe8zR3u3luDVVRjUEjz
jk4jE1cvAgMBAAECggEAQGVaHw2eap9LscjCcexQKQo4TmCt/fpi4ZiRTnOzu86j
rckhjdXcv+zUrhC4++6PBPZx7Wd1kbyw4ZFTg/RfrFWr9QVJrfotrzyWPW6UU+J1
v68MMn5fdkPsnXLVfu39pKs/ck6ZguOcv++bHLdxS/exI+S9oyi3bApH/R6HAP8e
qbLuzxrPSePB0N78Sycd2prvaWx5Jyitt45TJ47TBNUsaeYh7RSMR4+I9SNaGVi8
EPcLm/JKJQFprAN1nfgQcSCV5gij7Vm8n3QdCrSdFqRk74HYTnZczJpSvMKacr6A
gpjfdKLyNpXF2hu8gfscTHeIEa+ZM7zV+GPwwLlCsQKBgQD/sUyfs8XErpIbRVNz
LlOxJhw+vDr48NeT7vDGeRlKLlvDhNuRh7Y8qFpSb1dTZqy/K/dK25ussagDklF7
IV/syKwAHJVnpI+DU8mNlasdh2UCZqe5LOmkCBp5J1XKc9yCiuLl2nI79jxnuKU+
XGsALtWcqtsXWYtCuVbJgzP/XwKBgQDlTEj/27stPkmbKfCPF++XofYyixOFxKbM
BycklVVS2jdgSiwQJqY97pomdUoMw1VAMRqu91P21eENba6TYdStbHDBXSWv9yan
9JOmHn4BsX6apLZPJznYN6RpIpv36sudDZ6MqA/sSd0KUxxZN5nQPLEj6K/9PXnm
IEuMenJKMQKBgGilYZzC8J/sanOqzdqIDUylMk4ir2gO+cvbsMMp8MXBb2D30P0g
s97jcAPo+a5yiSS8k7sZYf8tvnunuJ1SLXY1Mt/g0uzWFr5C3myJ4rx1G0v+Q4qv
7PdrMZqAFMaE4Ky0tuQnoTUVM+enVEnPfSNYB6jU7FIEHdCzFefnQLK1AoGBAIjc
YJH+yBj4HZz0inYOVl6OOFHwuwVT3C7Zch6e15+tSfl6M1SsjPtYSD/+L+WwTVkU
NhYK393ryPln5xRnBzdAC4kOnOkLvtHu5JX5i0Jde/C3sW1LP3Zzt7ldA/UU+tVS
BLl1spNZfwqjkr8ZtcAaCa0iPLENmY2m5xEBIOhxAoGABv3x+vdrI3T1OtRA/uo0
JD+nl/U6T/+QPQqoZShz+s1MgIKOJRhtpp8BkuFIcxw8ezn2lJzFObDWhh5Y/6DO
cvyQwil3GTPSyc7b2DuPJtmry27MiJLuzIa9Pb1VoT9jDIMjPidhXNA3MrHs2ys9
LlijsWqMxILoBxJy8IKPU+0=
-----END PRIVATE KEY-----

CLIENT.CSR : La demande de certificat

-----BEGIN CERTIFICATE REQUEST-----
MIICzzCCAbcCAQAwgYkxCzAJBgNVBAYTAkZyMQswCQYDVQQIDAI5MzEVMBMGA1UE
BwwMVmlsbGV0YW5ldXNlMQwwCgYDVQQKDANpdXQxDTALBgNVBAsMBGdlaWkxEjAQ

BgNVBAMMCXN5cmluZV9uNDElMCMGCSqGSIb3DQEJARYWYWxsb3V0c2lyaW5lQGdt
YWlsLmNvbTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAOUFyxcF2SWK
NjVzLVjRIrNskOegNDdhQD6eZf9VUm/aGD94n9umUc8GHTM7vpOSQ6KGecqhmDo2
V19E6e+elQhHE7Oxb1t1Nh+GfafroW2ET9rV15vYNj6MZMsuS5WpTmV3u3MFk0IN
dOPr9EBpjcFt07EXFuLaGl06GthvXLs4s6ZM8++T0PKeA08xTFtigd402pnVBZyd
/7gF9yCMHV9gXC16JKkU8hCWFNYadQTn+d8284KDJC7ZD4T6nNvZAmNqafoFBsG8
ryadtfD9efvqaGmBW6biJYoz+QnihbbEkH/qKT32aaYTUzehF7zNHe7eW4NVVGNQ
SPOOTiMTVy8CAwEAAaAAMA0GCSqGSIb3DQEBCwUAA4IBAQCzLYUi4suPBTWNnyuv
bQtdNrpNrmkJs43V88GaHrPVLWOuxtAMTdVtvebOFnG6xAl2q9mLLC7RxgKvg1lb
teWjBOAv/pehAr2IF535D826gTOJsMC6w/SKmEM7xso4piT0ZAhe74FaWIeVVknY
XBuAQ54B8SsUps2IDXQcNrJq3dhboNRcQNamLxSvwvWYLS7KtkS0Awx7OFLFuPxC
yh2NO5zt8fE/YQsSSLB3ide8iWJJW8ATmiOAryZMccstDRBRoEPpJeTr45E9WKno
zBibZaoLOQUCYfxH4qQ5mwWtDd30JqZHgXnRKkjV8ZQ6fpVU8AYVLgTlyKlb7g3Y
K5vU
-----END CERTIFICATE REQUEST-----

CLIENT.CRT : Le certificat client

-----BEGIN CERTIFICATE-----
MIIDsDCCApigAwIBAgIBADANBgkqhkiG9w0BAQsFADCBkDELMAkGA1UEBhMCR0Ix
FzAVBgNVBAgMDlVuaXRlZCBLaW5nZG9tMQ4wDAYDVQQHDAVEZXJieTESMBAGA1UE
CgwJTW9zcXVpdHRvMQswCQYDVQQLDAJDQTEWMBQGA1UEAwwNbW9zcXVpdHRvLm9y
ZzEfMB0GCSqGSIb3DQEJARYQcm9nZXJAYXRjaG9vLm9yZzAeFw0yNjAxMDUxMDQ2
NDFaFw0yNjA0MDUxMDQ2NDFaMIGJMQswCQYDVQQGEwJGcjELMAkGA1UECAwCOTMx
FTATBgNVBAcMDFZpbGxldGFuZXVzZTEMMAoGA1UECgwDaXV0MQ0wCwYDVQQLDARn
ZWlpMRIwEAYDVQQDDAlzeXJpbmVfbjQxJTAjBgkqhkiG9w0BCQEWFmFsbG91dHNp
cmluZUBnbWFpbC5jb20wggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQDl
BcsXBdklijY1cy1Y0SKzbJDnoDQ3YUA+nmX/VVJv2hg/eJ/bplHPBh0zO76TkkOi
hnnKoZg6NldfROnvnpUIRxOzsW9bdTYfhn2n66FthE/a1deb2DY+jGTLLkuVqU5l
d7tzBZNCDXTj6/RAaY3BbdOxFxbi2hpdOhrYb1y7OLOmTPPvk9DyngNPMUxbYoHe
NNqZ1QWcnf+4BfcgjB1fYFwteiSpFPIQlhTWGnUE5/nfNvOCgyQu2Q+E+pzb2QJj
amn6BQbBvK8mnbXw/Xn76mhpgVum4iWKM/kJ4oW2xJB/6ik99mmmE1M3oRe8zR3u
3luDVVRjUEjzjk4jE1cvAgMBAAGjGjAYMAkGA1UdEwQCMAAwCwYDVR0PBAQDAgXg
MA0GCSqGSIb3DQEBCwUAA4IBAQCsx6WOpXdCr8Y/dBlcwo3qRU+O66niOO/CftHQ
mz4laLitkT0S3q8D75F2FBr5Fnlqp5kWkPBYutw1F7kQ93Z40WpbxXVBRgl5yVrn
ijk2uLM8H8yauTgnq8mcedwwg2RIxAFI7Cfr0VK4fQ/9VXE2hIU2CWTMGy8T7KAN
B9dK978EwZsZ+qDM2JrTvCrWkaZDplm2mRtb2BegK9l466aS0P5dZyOiWtdScRmQ
IGvdbOmVO0SDpWT38gRZOY6kEpFofewpMZN8fKcG81ZJ0qmUgI1EBU5i0AVmGOWc
g0xRoG6FrobGqOyDjW76cadjHN0c3q5eKqnJYu/b4oG6H+OB
-----END CERTIFICATE-----

MOSQUITTO.ORG.CRT : Le certificat CA Mosquitto

-----BEGIN CERTIFICATE-----

MIIEAzCCAuugAwIBAgIUBY1hlCGvdj4NhBXkZ/uLUZNILAwwDQYJKoZIhvcNAQEL

BQAwgZAxCzAJBgNVBAYTAkdCMRcwFQYDVQQIDA5Vbml0ZWQgS2luZ2RvbTEOMAwG

A1UEBwwFRGVyYnkxEjAQBgNVBAoMCU1vc3F1aXR0bzELMAkGA1UECwwCQ0ExFjAU

BgNVBAMMDW1vc3F1aXR0by5vcmcxHzAdBgkqhkiG9w0BCQEWEHJvZ2VyQGF0Y2hv

by5vcmcwHhcNMjAwNjA5MTEwNjM5WhcNMzAwNjA3MTEwNjM5WjCBkDELMAkGA1UE

BhMCR0IxFzAVBgNVBAgMDlVuaXRlZCBLaW5nZG9tMQ4wDAYDVQQHDAVEZXJieTES

MBAGA1UECgwJTW9zcXVpdHRvMQswCQYDVQQLDAJDQTEWMBQGA1UEAwwNbW9zcXVp

dHRvLm9yZzEfMB0GCSqGSIb3DQEJARYQcm9nZXJAYXRjaG9vLm9yZzCCASIwDQYJ

KoZIhvcNAQEBBQADggEPADCCAQoCggEBAME0HKmIzfTOwkKLT3THHe+ObdizamPg

UZmD64Tf3zJdNeYGYn4CEXbyP6fy3tWc8S2boW6dzrH8SdFf9uo320GJA9B7U1FW

Te3xda/Lm3JFfaHjkWw7jBwcauQZjpGINHapHRlpiCZsquAthOgxW9SgDgYlGzEA

s06pkEFiMw+qDfLo/sxFKB6vQlFekMeCymjLCbNwPJyqyhFmPWwio/PDMruBTzPH

3cioBnrJWKXc3OjXdLGFJOfj7pP0j/dr2LH72eSvv3PQQFl90CZPFhrCUcRHSSxo

E6yjGOdnz7f6PveLIB574kQORwt8ePn0yidrTC1ictikED3nHYhMUOUCAwEAAaNT

MFEwHQYDVR0OBBYEFPVV6xBUFPiGKDyo5V3+Hbh4N9YSMB8GA1UdIwQYMBaAFPVV

6xBUFPiGKDyo5V3+Hbh4N9YSMA8GA1UdEwEB/wQFMAMBAf8wDQYJKoZIhvcNAQEL

BQADggEBAGa9kS21N70ThM6/Hj9D7mbVxKLBjVWe2TPsGfbl3rEDfZ+OKRZ2j6AC

6r7jb4TZO3dzF2p6dgbrlU71Y/4K0TdzIjRj3cQ3KSm41JvUQ0hZ/c04iGDg/xWf

+pp58nfPAYwuerruPNWmlStWAXf0UTqRtg4hQDWBuUFDJTuWuuBvEXudz74eh/wK

sMwfu1HFvjy5Z0iMDU8PUDepjVolOCue9ashlS4EB5IECdSR2TItnAIiIwimx839

LdUdRudafMu5T5Xma182OC0/u/xRlEm+tvKGGmfFcN0piqVl8OrSPBgIIb+1IKJE

m/XriWr/Cq4h/JfB7NTsezVslgkBaoU=

-----END CERTIFICATE-----